

Contenido

La Regla de Oro: try-with-resources	2
1. El estilo antiguo y peligroso (¡No lo uses!)	2
2. La solución moderna: try-with-resources.....	3
El método correcto y actual:	3
¿Por qué usar siempre try-with-resources?	3
NIO.2 (java.nio.file): El Estándar Moderno en Java	4
Guía para Principiantes: Crear, Escribir, Leer y Borrar	4
1. Crear un Archivo	4
2. Escribir en un Archivo	5
3. Leer de un Archivo.....	5
4. Borrar un Archivo	6
Resumen.....	7

La Regla de Oro: try-with-resources

Antes de profundizar en NIO.2, es fundamental establecer una regla innegociable. Los flujos de datos (Streams) son **recursos del sistema**. Si abres uno, **debes cerrarlo** para evitar fugas de memoria (resource leaks) que podrían colapsar tu aplicación.

En las versiones antiguas de Java, esto se hacía en un bloque finally, lo cual resultaba en un código farragoso y propenso a errores.

1. El estilo antiguo y peligroso (¡No lo uses!)

Como se observa en los métodos tradicionales, cerrar un fichero requería anidar bloques try-catch dentro de un finally para asegurar que el método `.close()` se ejecutara incluso si ocurría un error previo.

```
BufferedReader br = null;
try {
    File file = new File("Ejemplo3.txt");
    FileReader fr = new FileReader(file);
    br = new BufferedReader(fr);

    String line;
    while ((line = br.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    System.err.println("Error: " + e.getMessage());
} finally {
    // Código engorroso para cerrar el recurso manualmente
    try {
        if (br != null) br.close();
    } catch (IOException ex) {
        System.err.println("Error al cerrar: " + ex.getMessage());
    }
}
```

2. La solución moderna: try-with-resources

Desde Java 7, la sentencia **try-with-resources** hace que este proceso sea limpio, seguro y automático. Cualquier objeto que implemente la interfaz `java.lang.AutoCloseable` (como todos los flujos de I/O de Java) puede usarse aquí.

El compilador genera automáticamente el bloque `finally` y llama al método `.close()` por ti, incluso si ocurre una excepción.

El método correcto y actual:

En este ejemplo, al declarar el `BufferedReader` y el `FileReader` dentro del paréntesis del `try`, Java garantiza que ambos se cerrarán correctamente al terminar el bloque.

```
// Cierra automáticamente los recursos, ocurran errores o no
File file = new File("Ejemplo3.txt");

try (FileReader fr = new FileReader(file);
    BufferedReader br = new BufferedReader(fr)) {

    String line;
    while ((line = br.readLine()) != null) {
        System.out.println(line);
    }
    // No hace falta br.close() ni fr.close(), ¡Java lo hace solo!

} catch (IOException e) {
    System.err.println("Ocurrió un error: " + e.getMessage());
}
```

¿Por qué usar siempre try-with-resources?

- **Código más limpio:** Elimina la necesidad de bloques `finally` repetitivos.
- **Seguridad:** Evita que olvides cerrar un fichero, lo cual es crítico cuando manejas muchos archivos a la vez.
- **Gestión de excepciones:** Si el cuerpo del `try` y el método `.close()` lanzan excepciones, Java gestiona correctamente las "excepciones suprimidas", permitiéndote ver la causa real del error.

Nota: Aunque en el sistema antiguo se usaba `Scanner.close()` o `BufferedWriter.flush()` de forma manual, la recomendación actual es delegar siempre esta responsabilidad al **try-with-resources**.

NIO.2 (java.nio.file): El Estándar Moderno en Java

A diferencia del sistema antiguo basado en la clase `File` y flujos bloqueantes (`java.io`), **NIO.2** (introducido en Java 7) es el sustituto eficiente para casi todas las operaciones. Es más potente, flexible y maneja errores mediante excepciones ricas y específicas.

- **Path:** Una representación del archivo independiente del sistema operativo (sustituye a `File`).
- **Paths:** Clase de utilidad para obtener rutas (ej. `Paths.get("datos.txt")`).
- **Files:** El motor principal con métodos estáticos para operar sobre archivos.

Guía para Principiantes: Crear, Escribir, Leer y Borrar

1. Crear un Archivo

En NIO.2 definimos la ubicación con `Path` y ejecutamos la creación. A diferencia del sistema antiguo, podemos crear directorios padres de forma sencilla.

```
Path ruta = Paths.get("documentos/notas.txt");

try {
    // Crea carpetas si no existen (similar a mkdir() pero más potente)
    Files.createDirectories(ruta.getParent());

    // Crea el archivo vacío
    Files.createFile(ruta);
    System.out.println("Archivo creado con éxito.");
} catch (IOException e) {
    System.err.println("Error al crear: " + e.getMessage());
}
```

2. Escribir en un Archivo

Olvidate del complejo `BufferedWriter` manual para tareas rápidas. `Files.write()` puede crear y escribir en un solo paso.

```
Path ruta = Paths.get("compras.txt");
String contenido = "Leche\nPan\nHuevos\n";

try {
    // Escribir contenido (sobrescribe por defecto)
    Files.write(ruta, contenido.getBytes());

    // Añadir contenido al final (Append)
    String extra = "Queso\n";
    Files.write(ruta, extra.getBytes(), StandardOpenOption.APPEND);
} catch (IOException e) {
    System.err.println("Error de escritura: " + e.getMessage());
}
```

3. Leer de un Archivo

Tu PDF muestra el uso de `Scanner` y `BufferedReader`. `NIO.2` permite leer todo de golpe o usar **Streams** para archivos grandes.

```
Path ruta = Paths.get("compras.txt");

try {
    // Opción A: Leer todo en una lista de Strings (ideal para archivos pequeños)
    List<String> lineas = Files.readAllLines(ruta);
    lineas.forEach(l -> System.out.println("Item: " + l));

    // Opción B: Stream para archivos grandes (memoria eficiente)
    try (Stream<String> stream = Files.lines(ruta)) {
        stream.filter(l -> !l.isEmpty()).forEach(System.out::println);
    }
} catch (IOException e) {
    System.err.println("Error de lectura: " + e.getMessage());
}
```

4. Borrar un Archivo

Sustituimos el método `file.delete()` por uno más seguro que no falla si el archivo no existe.

```
Path ruta = Paths.get("notas.txt");

try {
    boolean borrado = Files.deleteIfExists(ruta);
    System.out.println(borrado ? "Borrado" : "No encontrado");
} catch (IOException e) {
    System.err.println("Error al borrar: " + e.getMessage());
}
```

Resumen

Java ofrece un potente sistema de **manejo de archivos** a través de los paquetes *java.io* (clásico, basado en streams) y *java.nio* (moderno, más eficiente). Es recomendable usar el bloque *try-with-resources* para asegurar que los ficheros se cierren automáticamente.

Nombre	Representación	Sintaxis
Moderno (java.nio)		
<code>readString()</code>	Lee todo el contenido a un String	<code>Files.readString(path)</code>
<code>readAllLines()</code>	Lee todas las líneas a una List	<code>Files.readAllLines(path)</code>
<code>writeString()</code>	Escribe un String en un archivo	<code>Files.writeString(path)</code>
<code>exists()</code>	Comprueba si un archivo existe	<code>Files.exists(path)</code>
<code>createDirectory()</code>	Crea un directorio	<code>Files.createDirectory(p)</code>
<code>delete()</code>	Elimina	<code>Files.delete(path)</code>
Clásico (java.io)		
<code>FileReader</code>	Lee un archivo de texto	<code>new FileReader(file)</code>
<code>BufferedReader</code>	Lee texto de forma eficiente	<code>new BufferedReader(reader)</code>
<code>readLine()</code>	Lee una línea	<code>bufferedReader.readLine()</code>
<code>FileWriter</code>	Escribe en un archivo	<code>new FileWriter(file)</code>
<code>BufferedWriter</code>	Escribe texto de forma eficiente	<code>new BufferedWriter(writer)</code>
<code>write()</code>	Escribe una cadena	<code>bufferedWriter.write(text)</code>